

SENAI – Hermenegildo Campos De Almeida

ETAPA 3

DESENVOLVIMENTO DA API, INTEGRAÇÃO COM BD E GOOGLE SHEETS

ERICK PINHEIRO DE MELO

SAMUEL SILVA PAULINO

IGOR IAGO DE JESUS GONÇALVES

PROFESSOR: WILLIAM

Guarulhos
2025

SUMÁRIO

1. Desenvolvimento da API.....	3
2. Processamento e validação dos dados.....	4
3. Integração com bando de dados.....	4
4. Integração com Google Sheets.....	5
5. Segurança inicial do sistema.....	5
6. Testes e erros encontrados.....	6

1. DESENVOLVIMENTO DA API

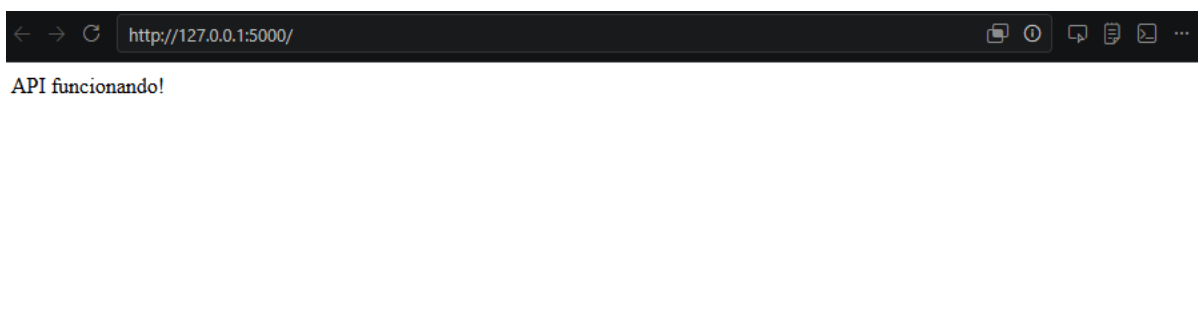
O que fizemos e por quê

Nessa etapa a gente criou a parte principal do sistema, que é a API. Ela basicamente é quem recebe os dados que vêm do ESP8266, trata essas informações e manda para os outros lugares (banco de dados e Google Sheets).

A gente usou **Python com Flask**, porque é mais simples de montar, rápido de configurar e funciona bem com JSON, que é o formato que o ESP envia os dados.

Resumo do conceito

- API REST: é uma forma de comunicação entre sistemas via internet, usando HTTP
- GET: usamos pra testar rotas e respostas simples
- POST: usado pra mandar os dados dos sensores
- JSON: formato dos dados (tipo chave e valor)
- Cliente/Servidor: o ESP manda os dados (cliente) e a API recebe e processa (servidor)



2. PROCESSAMENTO E VALIDAÇÃO DOS DADOS

O que a API faz com os dados

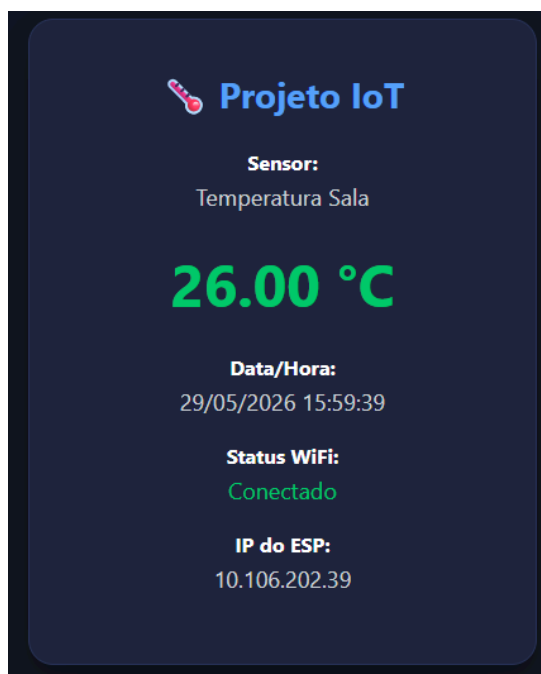
Quando os dados chegam do ESP, a API não aceita tudo direto. Ela primeiro verifica se está tudo certo (temperatura, rotação, estado do sistema etc). Se faltar alguma informação ou vier algo errado, a API não salva nada e já retorna erro.

3. INTEGRAÇÃO COM BANCO DE DADOS

Como funciona

Depois que a API recebe e valida os dados, ela manda tudo pro banco de dados usando pyodbc. Sempre que chega um dado válido, ele é salvo automaticamente no SQL Server. É salvo a temperatura, rotação, estado do sistema, alertas e logs.

```
SQL Server conectado!
Tabela verificada!
Google Sheets conectado!
* Serving Flask app 'main'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://10.106.202.32:5000
Press CTRL+C to quit
* Restarting with stat
```



4. INTEGRAÇÃO COM GOOGLE SHEETS

Pra que serve

Aqui a ideia foi deixar os dados visíveis em tempo real no Google Sheets, pra facilitar visualização e acompanhamento.

A API manda os dados direto pra planilha usando a API do Google Cloud.

Importante entender

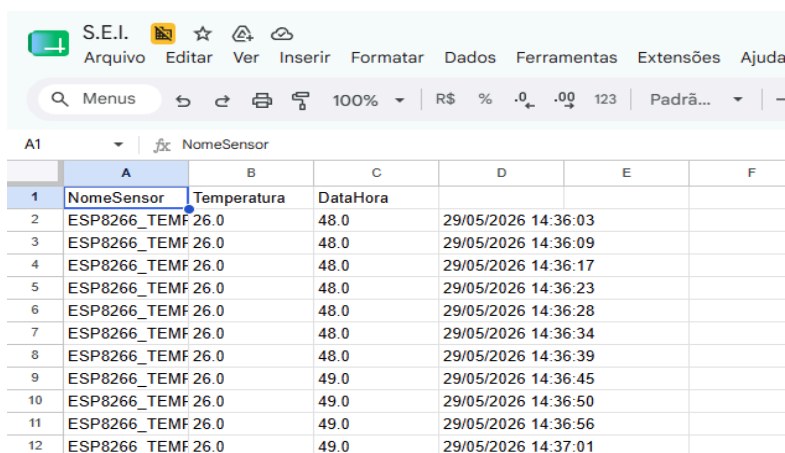
Google Sheets NÃO substitui banco de dados

Ele serve mais pra visualização

Banco de dados continua sendo o principal armazenamento

Se der problema

Se cair internet ou falhar conexão com o Google, o sistema não perde os dados. Ele salva no banco primeiro e só depois tenta mandar pra nuvem.



The screenshot shows the Google Sheets interface with a table containing sensor data. The table has columns for sensor name, temperature, and timestamp. The data is as follows:

	A	B	C	D	E	F
1	NomeSensor	Temperatura	DataHora			
2	ESP8266_TEMF 26.0	48.0	29/05/2026 14:36:03			
3	ESP8266_TEMF 26.0	48.0	29/05/2026 14:36:09			
4	ESP8266_TEMF 26.0	48.0	29/05/2026 14:36:17			
5	ESP8266_TEMF 26.0	48.0	29/05/2026 14:36:23			
6	ESP8266_TEMF 26.0	48.0	29/05/2026 14:36:28			
7	ESP8266_TEMF 26.0	48.0	29/05/2026 14:36:34			
8	ESP8266_TEMF 26.0	48.0	29/05/2026 14:36:39			
9	ESP8266_TEMF 26.0	49.0	29/05/2026 14:36:45			
10	ESP8266_TEMF 26.0	49.0	29/05/2026 14:36:50			
11	ESP8266_TEMF 26.0	49.0	29/05/2026 14:36:56			
12	ESP8266_TEMF 26.0	49.0	29/05/2026 14:37:01			

5. SEGURANÇA INICIAL DO SISTEMA

O que a gente fez

Ainda é uma segurança básica, mas já tomamos alguns cuidados:

Credenciais ficam em arquivos separados (não expostos no código principal)

Evitamos deixar API aberta pra qualquer pessoa

Sistema rodando só em rede local nos testes

Por que isso importa

Porque em IoT, se alguém invadir o sistema, pode alterar dados de sensores ou até gerar comandos errados. Isso em ambiente industrial pode virar um problema sério.

6. TESTES E ERROS ENCONTRADOS

Testes feitos

A gente testou tudo do começo ao fim:

- ESP conectando no Wi-Fi
- Envio de dados
- Recebimento pela API
- Salvamento no banco
- Envio pro Google Sheets

```

VERIFIQUE:
- Flask aberto
- IP correto
- Firewall Windows
- Mesmo Wi-Fi

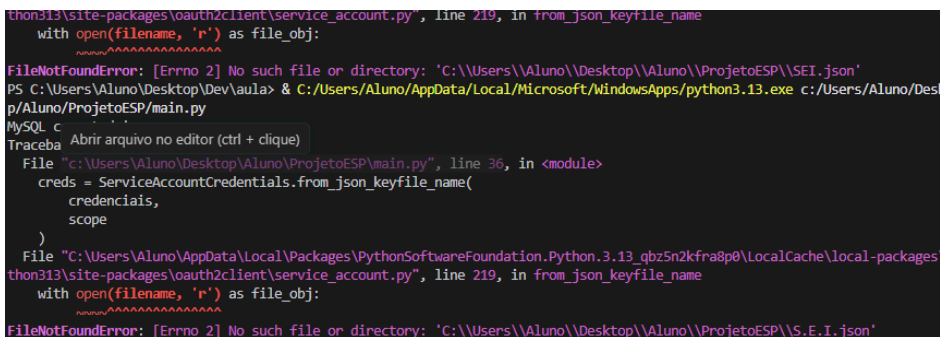
===== DADOS =====
Sensor      : ESP8266_TEMP
Temperatura : 24.9 C
Umidade     : 55.0 %
=====

Enviando dados...
http://10.106.202.32:5000/S.E.I.?sensor=ESP8266_TEMP&temperatura=24.9&umidade=55.0

```

Erro encontrado

Durante os testes com Google Sheets, deu um erro de arquivo de credencial



```

thon313\site-packages\oauth2client\service_account.py', line 219, in from_json_keyfile_name
    with open(filename, 'r') as file_obj:
        ~~~~~^
FileNotFoundError: [Errno 2] No such file or directory: 'C:\\Users\\Aluno\\Desktop\\Aluno\\ProjetoESP\\S.E.I.json'
PS C:\Users\Aluno\Desktop\Dev\aula> & C:/Users/Aluno/AppData/Local/Microsoft/WindowsApps/python3.13.exe c:/Users/Aluno/Desktop/Aluno/ProjetoESP/main.py
MySQL c
Traceba Abrir arquivo no editor (ctrl + clique)
File "C:\Users\Aluno\Desktop\Aluno\ProjetoESP\main.py", line 36, in <module>
    creds = ServiceAccountCredentials.from_json_keyfile_name(
        credenciais,
        scope
    )
File "C:\Users\Aluno\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\Python313\site-packages\oauth2client\service_account.py", line 219, in from_json_keyfile_name
    with open(filename, 'r') as file_obj:
        ~~~~~^
FileNotFoundError: [Errno 2] No such file or directory: 'C:\\Users\\Aluno\\Desktop\\Aluno\\ProjetoESP\\S.E.I.json'

```

O que aconteceu

O sistema estava procurando o arquivo “S.E.I.json”, mas ele estava com nome diferente e no lugar errado.

Como resolvemos

Pra resolver, a gente apenas renomeou o arquivo corretamente, colocamos na pasta certa e depois testamos de novo e deu certo, era um erro bobo, mas que importava muito.